

Project - Low Level Design

On

Deploy Automotive App to Kubernetes

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	Ameya Dole	EN22CS301111
2	Amishi Talwar	EN22CS301112
3	Ansh Mundra	EN22CS301142
4	Anuj Devnani	EN22CS301164
5	Arpith Shukla	EN22CS301203

Group Name: Group 04D2

Project Number: DO-16

Industry Mentor Name:

University Mentor Name: Prof. Akshay Saxena

Academic Year: 2025-26

Table of Contents

S.No.	Contents	Page No.
1.	Introduction.	3
1.1	Scope of the document.	3
1.2	Intended Audience	3
1.3	System overview.	3
2.	System Design.	4
2.1	Application Design	4
2.2	Process Flow.	5
2.3	Information Flow.	7
2.4	Components Design	8
2.5	Key Design Considerations	8
2.6	API Catalogue.	8
3.	References	9

1. INTRODUCTION

1.1 Scope of the Document

This document defines the architecture, deployment strategy, and operational workflow for deploying an Automotive Python Flask application onto a Kubernetes cluster using Minikube. It focuses on containerization, orchestration, CI/CD integration, and high-availability deployment techniques such as rolling updates to ensure zero downtime.

Key Scope Points of this Document are as follows:

- Covers end-to-end deployment from code (Git) to Kubernetes cluster
- Defines Docker-based containerization of the Flask application
- Includes Kubernetes manifests (Deployment, Service) for workload management
- Describes CI/CD pipeline using Jenkins or GitHub Actions
- Implements rolling updates for seamless version upgrades

1.2 Intended Audience

This document is intended for individuals involved in development, deployment, and operations of cloud-native applications. It assumes a foundational understanding of DevOps practices, containerization, and web application frameworks.

Target Audience:

- DevOps Engineers managing CI/CD pipelines and Kubernetes clusters
- Backend Developers working with Python Flask applications
- Cloud Engineers handling container orchestration and infrastructure
- Students and learners implementing real-world DevOps projects
- System Administrators responsible for deployment and monitoring

1.3 System Overview

The system consists of a Flask-based Automotive web application deployed as a containerized service on a Kubernetes cluster. The workflow integrates version control, automated build pipelines, container orchestration, and deployment strategies to ensure scalability, reliability, and zero downtime during updates.

System Components:

- **Source Control:** Git repository storing Flask application code

- **CI/CD Pipeline:** Jenkins or GitHub Actions for build, test, and deployment automation
- **Containerization:** Docker images for consistent runtime environments
- **Orchestration Layer:** Kubernetes (Minikube) managing pods, deployments, and services
- **Deployment Strategy:** Rolling updates to ensure continuous availability
- **Service Exposure:** Kubernetes Services to expose the application to users

2. SYSTEM DESIGN

2.1 Application Design

The architectural diagram illustrates a containerized Flask application deployed on a Kubernetes cluster through an automated CI/CD pipeline. It highlights the transition from development (Developer pushing code to Git) to production, where a Kubernetes Service routes user traffic to containerized Pods. The design showcases a modern microservices approach by separating the Business Logic, API Layer, and Data Layer within a Docker environment. Finally, it incorporates a zero-downtime strategy using rolling updates, ensuring that the car catalog remains accessible to users even during new version deployments.

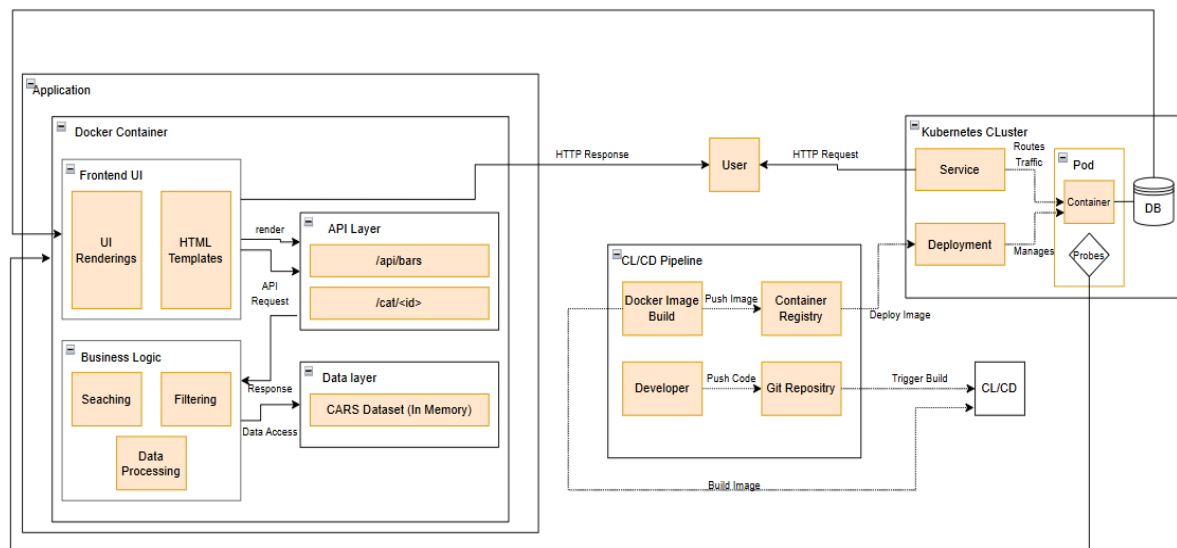


Figure 2.1: Application Design

2.2 Process Flow

The process begins with developing the Flask application and pushing the code to a Git repository, which triggers the CI/CD pipeline. The pipeline builds a Docker image and deploys it to the Kubernetes (Minikube) cluster. Kubernetes creates and manages pods, and the application is exposed to users through a service for external access. Users can interact with the application via a web browser, and any new updates trigger the pipeline again for seamless deployment. This cycle ensures continuous integration and zero-downtime operation of the application.

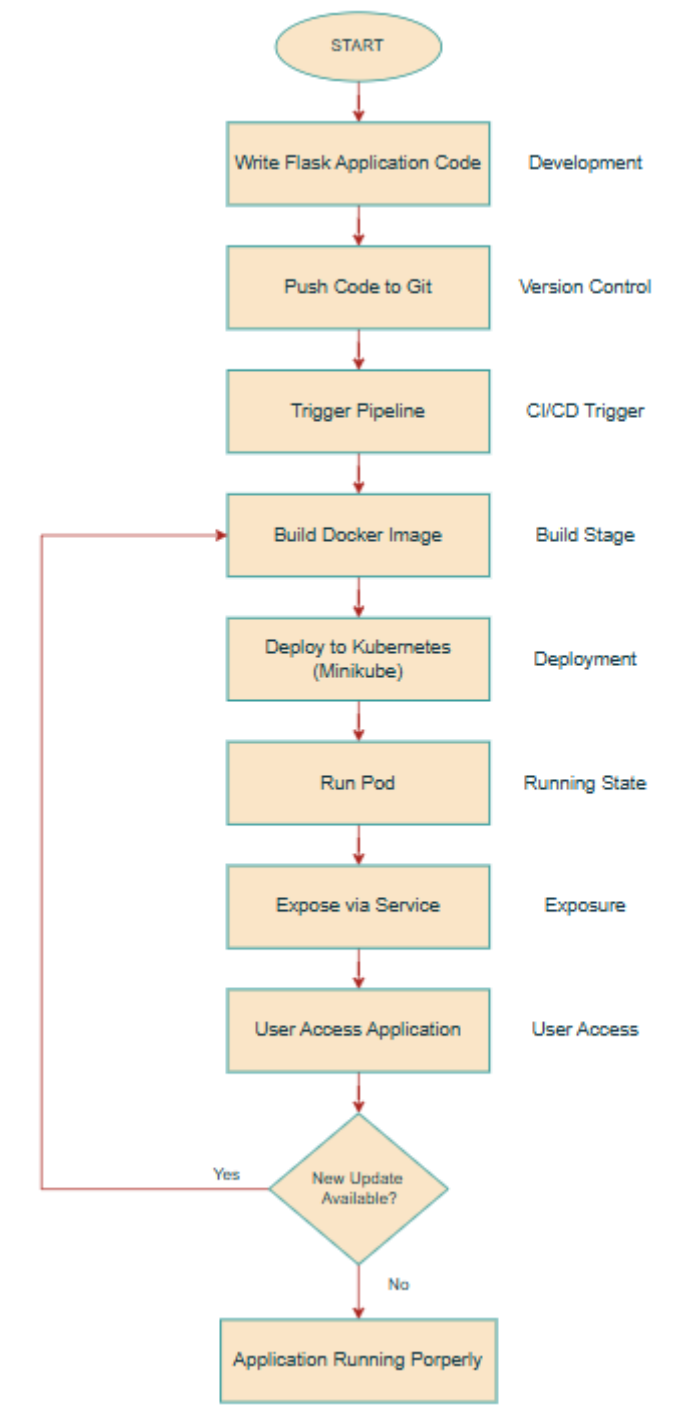


Figure 2.2: Process Flow

2.3 Information Flow

2.3.1 Runtime Flow

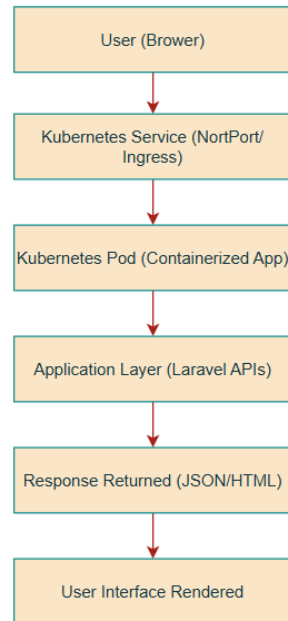


Figure 2.3: Runtime Flow

2.3.2 Deployment Flow

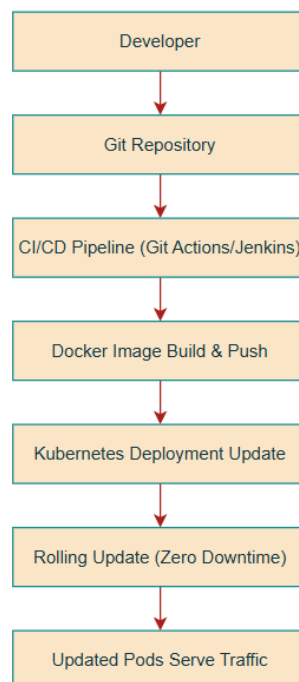


Figure 2.4: Deployment Flow

2.4 Components Flow

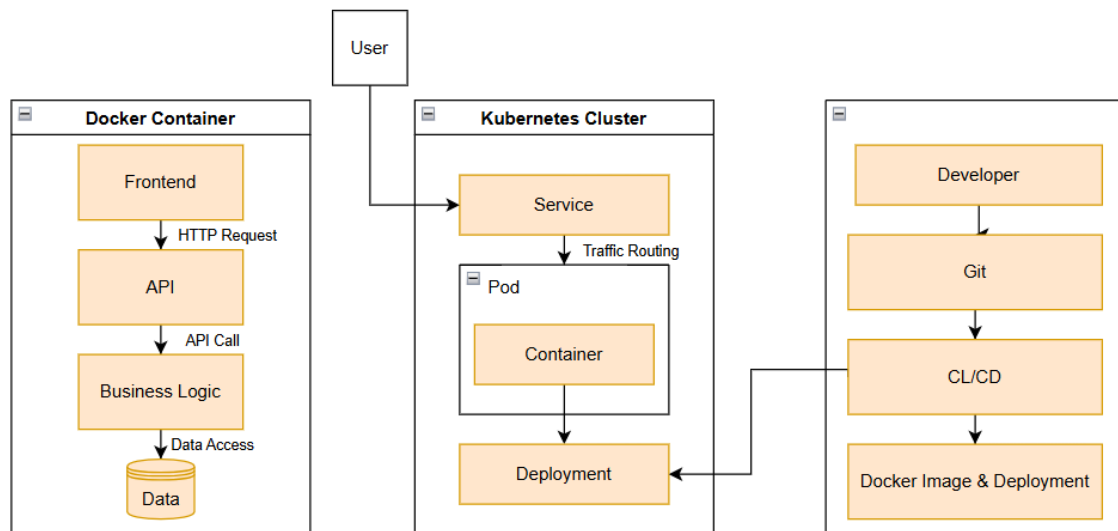


Figure 2.5: Component Design

2.5 Key Design Considerations

- Modular Flask-based architecture for maintainability and feature separation
- Docker containerization for consistent and portable deployment
- Kubernetes-based deployment ensuring scalability and high availability
- CI/CD pipeline automation for fast and reliable delivery
- Zero-downtime strategy using rolling updates (replicas, surge, availability control)

2.6 API Catalogue

1. **/api/cars (GET):** This is the main API endpoint used in the system. It returns a list of cars in JSON format. It supports optional query parameters like category and search to filter results dynamically. This API is used by the frontend to display and update car listings.
2. **/car/<car_id> (GET):** This endpoint fetches details of a specific car based on its ID. It returns an HTML page with full specifications. If the car is not found, it returns a 404 error.
3. **/ (GET):** This is the home route that loads the main UI. It sends all car data and categories to the frontend and renders the homepage.

3. REFERENCES

- Flask Official Documentation – <https://flask.palletsprojects.com>
- Docker Documentation – <https://docs.docker.com>
- Kubernetes Documentation – <https://kubernetes.io/docs>
- GitHub Actions Documentation – <https://docs.github.com/actions>
- Jenkins Documentation – <https://www.jenkins.io/doc>
- REST API Design Guide – <https://restfulapi.net>